

Tight Coupling, Loose Coupling & Dependency Injection (DI)

1. Why Coupling Matters in Real Projects

In any real-world application, we never work with a single class.

Applications are built using:

- Multiple classes
- Each class depending on others
- Objects collaborating to deliver functionality

The way these classes depend on each other **decides how flexible, testable, and maintainable** the application becomes.

This is where **coupling** comes into play.

2. What Is Tight Coupling?

Tight coupling occurs when:

- One class **directly creates** another class
- The dependency is **hard-coded**
- Changes in one class force changes in another

Real-life analogy

A mobile phone that supports **only one telecom provider**.

No freedom. No replacement. No flexibility.

3. Tight Coupling – Java Example

```
class AirtelSim {
    public void call() {
        System.out.println("Calling using Airtel SIM");
    }
}

class Mobile {
    AirtelSim sim = new AirtelSim(); // Tight coupling

    public void makeCall() {
        sim.call();
    }
}
```

Problems with this approach:

- Mobile is permanently tied to Airtel Sim
- Cannot switch to Jio or Idea without changing code
- Hard to test
- Violates good design principles

This is **tight coupling**.

4. Why Tight Coupling Is Dangerous in Enterprise Apps

In enterprise applications:

- Business requirements change frequently
- Implementations evolve
- Dependencies must be replaceable

Tight coupling leads to:

- Fragile code
- Difficult refactoring
- High maintenance cost

We need **flexibility**.

5. Loose Coupling – The Correct Approach

Loose coupling means:

- Classes depend on **abstractions**, not implementations
- Dependencies are **provided from outside**
- Implementation can change without breaking code

Real-life analogy

A mobile with a **SIM slot**.

The phone depends on a SIM — not on Airtel, Jio, or Idea specifically.

6. Introducing Abstraction (Interface)

```
interface Sim {  
    void call();  
}
```

Implementations:

```
class AirtelSim implements Sim {  
    public void call() {  
        System.out.println("Calling using Airtel SIM");  
    }  
}  
  
class JioSim implements Sim {  
    public void call() {  
        System.out.println("Calling using Jio SIM");  
    }  
}
```

7. Loose Coupling – Java Example

```
class Mobile {  
    private Sim sim;  
  
    public Mobile(Sim sim) {  
        this.sim = sim;  
    }  
  
    public void makeCall() {  
        sim.call();  
    }  
}
```

Usage:

```
Sim sim = new AirtelSim(); // or JioSim  
Mobile mobile = new Mobile(sim);  
mobile.makeCall();
```

Now:

- Mobile depends only on **Sim interface**
- Implementation can change at runtime
- No code modification needed inside Mobile

This is **loose coupling**.

8. What Is Dependency Injection (DI)?

Dependency Injection means:

Instead of creating dependencies inside a class, they are **injected from outside**.

In simple terms:

- **Don't use new inside business classes**

- Let someone else provide the dependency
-

9. Types of Dependency Injection

1. Constructor Injection (Recommended)

```
class Mobile {  
    private Sim sim;  
  
    public Mobile(Sim sim) {  
        this.sim = sim;  
    }  
}
```

- ✓ Immutable
 - ✓ Mandatory dependency
 - ✓ Best for testing
-

2. Setter Injection

```
class Mobile {  
    private Sim sim;  
  
    public void setSim(Sim sim) {  
        this.sim = sim;  
    }  
}
```

- ✓ Optional dependency
 - ✓ Flexible
-

3. Field Injection (Not Recommended)

```
class Mobile {  
    @Autowired  
    private Sim sim;  
}
```

- ✗ Hard to test
 - ✗ Hidden dependencies
-

10. How Spring Fits In

Spring:

- Creates objects
- Decides which implementation to inject
- Wires dependencies automatically
- Manages lifecycle

You **declare what you need**, Spring provides **how it is created**.

This is where:

- Loose coupling
 - Dependency Injection
 - Inversion of Control
- come together.
-

11. Interview Perspective (Very Important)

If asked:

“How does Spring reduce tight coupling?”

Best answer:

By enforcing programming to interfaces and injecting dependencies at runtime using Dependency Injection, thereby achieving loose coupling through Inversion of Control.
